

## 16 Ein- und Ausgabe



Programme laufen fast immer nach dem Schema „Eingabe/Verarbeitung/Ausgabe“ (EVA-Prinzip) ab. Schon daran ist die Wichtigkeit der **Ein- und Ausgabe** von Programmen zu erkennen.

Ein gutes Ein-/Ausgabe-System ist für die Akzeptanz einer Programmiersprache von großer Bedeutung.



Die Ein- und Ausgabe in einem Programm kann unterschiedlichen Zwecken dienen. Die wichtigsten Fälle sind: Kommunikation mit einem Anwender über die Tastatur, den Bildschirm etc. und Zugriff auf die Daten, die in einem Dateisystem auf einem Massenspeicher langfristig gespeichert sind.

Für die Ein- und Ausgabe stellt C eine Standardbibliothek zur Verfügung. Durch Einbinden der Standard-Bibliothek `<stdio.h>` kann auf eine große Anzahl nützlicher Prototypen von Ein- und Ausgabefunktionen zugegriffen werden, die auf dem Konzept von Datenströmen (auf Englisch „streams“) basieren.



Ein **Datenstrom** kann eine Datenquelle wie eine Tastatur beziehungsweise ein Datenziel wie einen Bildschirm mit dem Programm verbinden. Mit Hilfe eines Datenstroms kann ein Programm aber auch mit einer Datei verbunden werden. Dateien können dabei sowohl Datenquelle als auch Datenziel sein.

Das Architekturprinzip von UNIX, nämlich Dateien und Peripheriegeräte über Datenströme in gleicher Weise zu behandeln, ist ebenso auch in C zu finden.



Es muss hier noch erwähnt werden, dass die Sprache C keinen Standard für grafische Oberflächen definiert und dass solche Programme nur über zusätzliche, meist betriebssystem- und/oder compilerabhängige Bibliotheken für grafische Oberflächen erstellt werden können. Darauf kann im Rahmen dieses Buches nicht eingegangen werden.

---

**Ergänzende Information** Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann [https://doi.org/10.1007/978-3-658-45209-4\\_16](https://doi.org/10.1007/978-3-658-45209-4_16).

## 16.1 Speicherung von Daten in Dateisystemen

Bei der Datenverarbeitung ist die automatische Verarbeitung von Daten mit Hilfe von Programmen das Hauptziel. Programme, die vom Prozessor bearbeitet werden sollen, müssen für die Dauer ihrer Abarbeitung in den Hauptspeicher geladen werden. Die dauerhafte Speicherung der Programme und ihrer Daten erfolgt auf Massenspeichern.

Massenspeicher wie zum Beispiel eine Festplatte werden mit Hilfe des Betriebssystems angesprochen und verwaltet. Die Komponente des Betriebssystems, die dafür verantwortlich ist, ist das **Dateisystem** (auf Englisch „**file system**“).

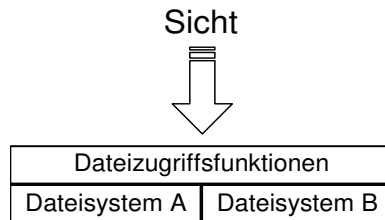


Nicht jedes Betriebssystem muss ein Dateisystem haben. In der Prozessdatenverarbeitung werden auch Betriebssysteme verwendet, die keine Massenspeicher verwalten. Bei Betriebssystemen, welche auf Massenspeicher zugreifen, konnte aufgrund der Vielfalt der Geräte, ihrer unterschiedlichsten Hardware-Eigenschaften und Verwaltungs-Mechanismen der Zugriff auf Dateien der peripheren Speicher früher im Allgemeinen nicht direkt durch die Programmiersprache erfolgen, sondern wurde durch Funktionen des Betriebssystems ausgeführt, die für die entsprechende Programmiersprache zur Verfügung gestellt wurden. Für die verschiedenartigen Massenspeicher wie Festplatten, Disketten, CDs, Memorysticks etc. wurden im Laufe der Zeit durch die Betriebssysteme zunehmend komfortablere Dateisysteme bereitgestellt.

Eine **Datei** (auf Englisch „**file**“) ist eine Zusammenstellung von logisch zusammengehörigen Daten, die als eine Einheit behandelt werden und unter einem dem Betriebssystem bekannten Dateinamen (file name) abgespeichert werden.



Erst durch UNIX, das Geräte als Dateien behandelte, und durch das Konzept der Datenströme wurde es möglich, von der Programmiersprache aus mit Dateizugriffsfunktionen direkt auf Peripheriegeräte zuzugreifen. Das spezielle Dateisystem bleibt dabei verborgen. Dies symbolisiert das folgende Bild:



Über einen Datenstrom kann man direkt auf die Festplatte schreiben oder von ihr lesen (siehe Kapitel [→ 16.6](#) und [→ 16.8](#)). Programme können damit leichter portiert werden, da sie nicht von den Interna des Dateisystems eines bestimmten Betriebssystems abhängen, sondern eine wohldefinierte Programmschnittstelle benutzen.

### 16.1.1 Dateien unter UNIX – das Streamkonzept

Die Programmiersprache C wurde geschrieben, um mit ihrer Hilfe das Betriebssystem UNIX zu implementieren. Die Architekturvorstellungen beim Entwurf von UNIX spiegeln sich deshalb auch in C wider.

Eine Datei unter UNIX ist ein einziger mit einem Namen versehener Datensatz beliebiger Länge, der aus einer Folge von Bytes besteht. Ein solcher Datensatz aus beliebig vielen Bytes wird im Englischen als Datenstrom (auf Englisch „stream“) bezeichnet.



Ein **Stream** ist eine Folge von Bytes. Die Bedeutung dieser Bytes (Buchstaben, Zahlen, Bitmuster) spielt dabei für die Verarbeitung durch das Dateisystem keine Rolle.



Da das Dateisystem als Struktureinheit nur Bytes kennt, bietet es keine Unterstützung für das Lesen und das Schreiben von strukturierten Informationen. Hierfür werden jedoch in C Bibliotheksfunktionen bereitgestellt.

Die Dateien des Dateisystems, die angesprochen werden, sind in UNIX also Folgen von Bytes oder Byte-Arrays.

Das Interpretieren eines Stroms von Bytes beim Lesen oder Schreiben hat nicht durch das Dateisystem, sondern durch den Anwender zu erfolgen.



Der Anwender nimmt die Interpretation des Bytestroms meist nicht selbst vor, sondern bedient sich hierzu der Bibliotheksfunktionen, die es beispielsweise erlauben, von einer Textdatei zeilenweise zu lesen oder in eine tabellarische Datei Datensätze fester Länge zu schreiben. Unter einer tabellarischen Datei wird hierbei eine Datei verstanden, bei welcher jeder Datensatz dieselbe Struktur aufweist, beispielsweise die Auflistung von Ein- und Ausgaben bei einer Buchhaltungs-Software.

Programme, die über Dateien kommunizieren, müssen sich über das Format, wie die entsprechende Datei beschrieben und gelesen wird, verständigen.



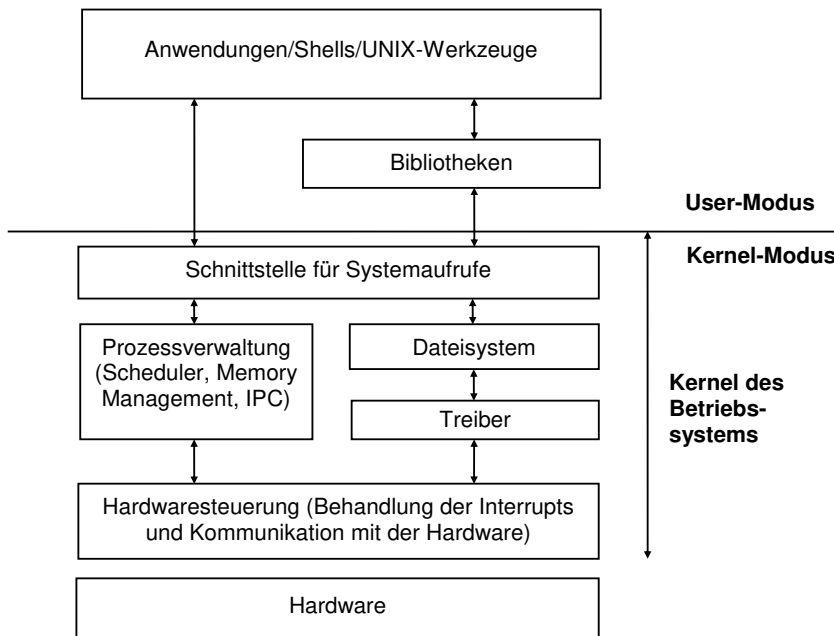
Die Umsetzung auf das byteweise arbeitende Dateisystem nehmen die Bibliotheksfunktionen vor. Die Umsetzung vom Dateisystem auf die eigentliche Geräthardware übernimmt dann ein Gerätetreiber (auf Englisch device driver).

### 16.1.2 Schichtenmodell von UNIX für die Ein- und Ausgabe

Ein von einem Anwender geschriebenes Programm greift nicht direkt auf die Ein-/Ausgabegeräte zu, sondern auf C-Bibliotheksfunktionen, die für die Ein-/Ausgabe vorgesehen sind. Diese wiederum rufen Systemfunktionen des Betriebssystems auf.



Das Besondere an UNIX ist, dass sämtliche Ein- und Ausgaben über ein Dateisystem im Kernel des Betriebssystems gehen – nicht nur die Ein-/Ausgaben von und zu der Festplatte, sondern generell für alle Peripheriegeräte wie Bildschirm, Tastatur und so fort. Im Folgenden ein Bild der Architektur von UNIX (gemäß [Bac91]):



Alle Ein-/Ausgaben eines Programms beziehen sich in UNIX auf Dateien im Dateisystem.



Die Geräteabhängigkeit ist bei UNIX durch das Dateisystem verborgen.



Die Geräte selbst werden durch Treiberprogramme (auf Englisch device driver) angesteuert. Der Kernel ist derjenige Teil des Betriebssystems, der in einem besonders geschützten Modus (Kernel-Modus) läuft, damit beispielsweise Programmierfehler eines Anwenders keinen Schaden anrichten können. Der Kommandointerpreter – unter UNIX Shell genannt – und UNIX-Werkzeuge sind auch Teile des Betriebssystems. Sie werden jedoch nicht im Kernel-Modus, sondern im User-Modus ausgeführt und haben damit denselben Schutz wie normale Programme.

In dieser Architektur lassen sich sowohl blockorientierte Geräte wie Festplatten als auch zeichenorientierte Schnittstellen wie Tastaturen oder Netzwerkschnittstellen in einfacher Weise einbinden. Jedes Gerät wird durch einen Namen, der wie ein Dateiname aussieht, angesprochen und die Ein- und Ausgabe erfolgt durch das Lesen von Dateien beziehungsweise Schreiben in Dateien, als ob Geräte gewöhnliche Dateien auf einer Festplatte wären.



Es ist sogar möglich und üblich, dass ein Gerät wie eine Festplatte oder eine SD-Karte zwei Gerätetreiber hat, eine Block- und eine Zeichenschnittstelle. Aber so tief – bis zu den Gerätetreibern – muss man nicht sehen. Man ruft einfach die C-Bibliotheksfunktionen auf, die auf das Dateisystem zugreifen. Was darunter liegt, ist verborgen.

## 16.2 Das Ein-/Ausgabe-Konzept von C

C bietet selbst keine Sprachmittel für die Ein- und Ausgabe. Für die Ein- und Ausgabe werden stattdessen Bibliotheksfunktionen verwendet, die jedoch wie die Sprache selbst standardisiert sind. Ihre Schnittstellen sind in der Bibliothek `<stdio.h>` aufgeführt.



Auf eine Datei wird in einem C-Programm über eine **Dateivariablen** zugegriffen. Diese ist entweder ein sogenannter **File-Pointer** (siehe Kapitel [→ 16.5](#)) oder eine systemspezifische Datei-Identifikation (siehe Anhang [→ B](#)).

Eine Datei liegt dabei normalerweise im Dateisystem und hat einen Namen, beispielsweise den Namen `test.dat`. Zur Laufzeit wird dann die Verknüpfung zwischen der Dateivariablen und der Datei auf der Festplatte hergestellt.

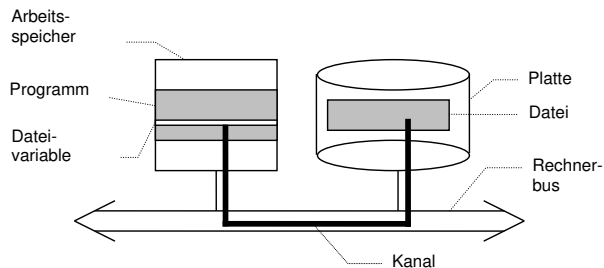
Beim Öffnen einer Datei muss dem Programm mitgeteilt werden, welche Dateivariablen des Programms mit welcher Datei des Dateisystems verknüpft werden soll.



Dies erfolgt beispielsweise durch folgende Anweisung:

```
filePointer = fopen("test.dat", "w");
```

Hier wird mit der Standardfunktion `fopen()` eine Datei `test.dat` zum Schreiben (w für write) geöffnet und mit der Dateivariablen `filePointer` verknüpft. `filePointer` stellt bildlich gesprochen den Kanal vom Programm zur Datei auf der Festplatte dar:



Über diesen Kanal `filePointer` kann in dem oben skizzierten Fall ("w") nur geschrieben werden. Prinzipiell kann jedoch über einen Kanal gelesen oder geschrieben werden.

### 16.2.1 Physische und logische Ebene

Beim Programmieren sieht man nur die logische Ebene einer Datei, in anderen Worten die Struktur ihrer Informationen und nicht die Blöcke auf der Festplatte. Die Informationen einer Datei, die man beim Programmieren als eine logische Einheit sieht, können physisch auf zahlreiche Plattenblöcke aufgeteilt sein. Diese Plattenblöcke einer Datei müssen nicht unbedingt sequenziell hintereinander auf der Festplatte liegen, sondern können über die Festplatte verstreut sein. Um die physischen Blöcke einer Festplatte braucht man sich nicht zu kümmern.

Die logische Struktur einer Datei wird auf die physischen Blöcke einer Festplatte durch das Dateisystem umgesetzt.



Die einzige Stelle, an der man im Programm noch das Dateisystem sieht, ist die Verknüpfung einer Dateivariablen des Programms, die den (logischen) Kanal zum Peripheriegerät repräsentiert, mit dem Namen einer Datei, der dem Dateisystem bekannt ist. Der Kanal zur Datei wird vom Betriebssystem zur Verfügung gestellt.

## 16.3 Standardeingabe und Standardausgabe in C

Vom Laufzeitsystem des C-Compilers werden drei Standardkanäle für die Standardeingabe, die Standardausgabe und die Standardfehlerausgabe in Form der globalen File-Pointer **stdin**, **stdout** und **stderr** zur Verfügung gestellt.



Das Laufzeitsystem initialisiert diese File-Pointer vor dem Programmstart. Normalerweise stellen diese Standardkanäle Kanäle zum Bildschirm beziehungsweise zur Tastatur dar. Sie können aber auch beim Programmstart mit Hilfe von Kommandos an das Betriebssystem auf Dateien oder andere Geräte umgelenkt werden. Diese File-Pointer stehen jedem C-Programm zur Verfügung und stellen ein wichtiges Hilfsmittel für die Ein- und Ausgabe dar.

Der Erfolg von UNIX hängt unter anderem auch mit seiner großen Flexibilität im Umgang mit Dateien zusammen. Hierbei haben sich zwei Konzepte als sehr nützlich erwiesen – weshalb sie auch von Windows übernommen wurden – nämlich die Umlenkung (Umleitung) der Ein- und Ausgabe und das Pipelining. Diese beiden Konzepte werden in den folgenden beiden Unterkapiteln genauer erläutert.

### 16.3.1 Umlenkung der Ein- und Ausgabe

Das **Umlenken** der Standardeingabe und Standardausgabe erfolgt auf der Kommandoebene des Betriebssystems unter Windows gleich wie unter UNIX, nämlich durch die Umlenk-Operatoren **>** und **<**.



Die Umlenkung der **Standardausgabe** erfolgt wie im folgenden Beispiel:

```
myprog > test.out
```

Die vom Programm `myprog` an die Standardausgabe geleiteten Zeichen werden in die Datei `test.out` geschrieben. Zum Beispiel werden alle Ausgaben von `printf()` jetzt in die Datei `test.out` geschrieben.



Von einer Umlenkung merkt ein Programm nichts.



Unter UNIX wie auch unter Windows wird durch `myprog > test.out` automatisch die gewünschte Datei erzeugt. Danach steht die Datei `test.out` im aktuellen Verzeichnis.

Die Umlenkung der **Standardeingabe** erfolgt wie im folgenden Beispiel:

```
myprog < test.inp
```

Damit kommen die Eingaben für das Programm `myprog` nicht mehr von der Tastatur, sondern aus der Datei `test.inp`. Auch hier merkt das Programm nichts davon. Es behandelt jedes Byte so, als wäre es von der Tastatur gekommen.

Beide Umlenkungsarten können auch kombiniert werden, das heißt, man kann Standardeingabe und Standardausgabe gleichzeitig umlenken:

```
myprog < test.inp > test.out
```

Die Eingabe erfolgt dann nicht mehr von der Tastatur und die Ausgabe geht nicht mehr an den Bildschirm.

Die Angaben zur Umlenkung der Standardeingabe und der Standardausgabe werden dabei nicht an das Programm weitergeleitet, sondern die Umlenkung wird vom Kommandointerpreter des Betriebssystems durchgeführt.

Damit Fehlermeldungen nach wie vor trotz erfolgter Umlenkung der Standardausgabe am Bildschirm erscheinen können, gibt es einen dritten, unabhängigen Standardkanal, die **Standardfehlerausgabe**. Auch diese kann mit Mitteln des Betriebssystems umgelenkt werden. Beispielsweise durch folgende Zeile:

```
myprog 2> errors.log
```

Die 2 entspricht dabei der internen Nummer des Kanals `stderr`. Der Kanal `stdin` hat die Nummer 0 und `stdout` hat die Nummer 1. Die Nummern 0, 1 und 2 haben bei allen Programmen genau diese Bedeutung.

### 16.3.2 Pipelining

Durch **Pipes** (Röhren) werden mehrere Programme in der Weise verbunden, dass die Standardausgabe des einen Programms als Standardeingabe des anderen Programms verwendet wird.



Das Pipelining erfolgt auf der Kommandoebene des Betriebssystems unter Windows gleich wie unter UNIX in der Form:

```
Programm1 | Programm2
```

Der senkrechte Strich bedeutet, dass die Ausgabe von Programm1 umgeleitet und zur Eingabe von Programm2 wird. Diese Syntax soll durch ein bekanntes Beispiel aus der UNIX-Welt illustriert werden. Die mittels einer Pipe verbundenen Kommandos

```
ls | sort
```

geben eine sortierte Liste der Dateien des aktuellen Verzeichnisses aus. Hierbei wird die Standardausgabe des Programms `ls` als Standardeingabe an das Programm `sort` gegeben.

Die Standardausgabe eines Programms wird durch den Pipe-Operator `|` zur Standardeingabe für das nächste Programm.



Der dabei verwendete Puffer, in den das eine Programm hineinschreibt und aus dem das andere Programm herausliest, wird als Pipe bezeichnet.



Der Mechanismus wird als Fließbandverarbeitung oder Pipelining bezeichnet. Der Vorteil ist zum einen, dass hierbei kein Zugriff zur Festplatte erforderlich ist, sondern nur zum Arbeitsspeicher. Im Arbeitsspeicher wird quasi eine temporäre Datei angelegt, die wieder automatisch gelöscht wird. Durch das Hintereinanderreihen von Programmen mit Hilfe von Pipes kann man mächtige Funktionen erzeugen. Die einzelnen Programme können dabei ziemlich klein sein.

Ein Programm, welches nur Daten von der Standardeingabe liest, diese verarbeitet und sein Resultat nur auf die Standardausgabe schreibt, wird als Filter bezeichnet. Verschiedene Filter können durch Pipes verbunden werden.



```
prog1 | filter1 | filter2 | prog2
```

### 16.3.3 Funktionen für die Standardeingabe und Standardausgabe

Wie man sieht, eröffnet die Programmierung mit der Standardeingabe und Standardausgabe sehr viele Möglichkeiten: Diese Programme können sowohl mit Bildschirm und Tastatur arbeiten, bei Bedarf – beispielsweise wenn sie im Hintergrund arbeiten sollen – ihre Daten aber auch aus Dateien beziehen oder ihre Ergebnisse in Dateien ablegen. Sie sind nicht von speziellen Dateinamen abhängig, sondern bei jedem Programmstart kann eine andere Datei in einer Umlenkung berücksichtigt werden. Einfache Programme können über Pipes zu komplexeren Operationen verknüpft werden, ohne dass in eines der Programme eingegriffen werden muss.

Wegen der herausragenden Bedeutung der Standardeingabe und Standardausgabe stellt die C-Bibliothek eigene Funktionen für die Standardeingabe und die Standardausgabe zur Verfügung.



Häufig sind diese nur als Makros implementiert, die vom Präprozessor auf Aufrufe von Funktionen des Betriebssystems umgesetzt werden. Sie stellen also nur Abkürzungen dar, da sie häufig benutzt werden. Ihre Existenz bedeutet nicht, dass Standardeingabe, Standardausgabe und „normale“ Dateien unterschiedlich behandelt werden.

## 16.4 High- und Low-Level-Bibliotheksfunktionen zur Ein- und Ausgabe

Die C-Bibliothek implementiert für die Ein- und Ausgabe das Stream-Konzept, welches von UNIX her bekannt ist. Eine Datei wird als ein Array von Zeichen betrachtet.



Arbeitet das Betriebssystem anders, so wird dies durch die Bibliotheksfunktionen von C verborgen.

Die Bibliotheksfunktionen zur Ein- und Ausgabe in C lassen sich unterteilen in High-Level-Funktionen und Low-Level-Funktionen.

### 16.4.1 High-Level-Funktionen

Die **High-Level-Funktionen** bieten eine einfache und portable Schnittstelle für den Umgang mit Dateien. Wie oben bereits erwähnt, gibt es spezielle Funktionen, die auf die Standardein- beziehungsweise -ausgabe zugreifen. Eine der bekanntesten Funktionen ist `printf()`.

`printf()` ist eine Abkürzung für „print formatted“ und dient zur formatierten Ausgabe auf die Standardausgabe.



Mit `printf()` können beispielsweise `int`-Zahlen formatiert – also in einer lesbaren Form – als Folge von Dezimalzeichen ausgegeben werden.

In der Standardbibliothek ist jedoch die Funktionalität von `printf()` auch für das Schreiben in Dateien verfügbar. Man fügt dem Namen der Funktion einfach ein `f` am Anfang hinzu.

Das erste `f` von `fprintf()` kommt von „file“. `fprintf()` dient also allgemein zur formatierten Ausgabe in irgend eine Datei, beziehungsweise irgend einen Stream.



Hierauf wird in den folgenden Kapiteln noch genauer eingegangen.

### 16.4.2 Low-Level-Funktionen

Low-Level-Funktionen werden auch elementare Ein-/Ausgabefunktionen genannt.

Die Low-Level-Funktionen für den Dateizugriff sind Bibliotheksfunktionen, die Aufrufe von Betriebssystemfunktionen bewirken.



Die High-Level-Dateizugriffsfunktionen der Standardbibliothek sind – zumindest unter UNIX – mit Hilfe von Aufrufen der Low-Level-Funktionen für den Dateizugriff implementiert.



Die Low-Level-Funktionen werden in Anhang → B beschrieben.

Low-Level-Funktionen bieten auch Möglichkeiten für die Ein- und Ausgabe, die es bei den High-Level-Funktionen nicht gibt.

Die Low-Level-Funktionen haben jedoch den Nachteil, dass sie nicht unabhängig vom Betriebssystem standardisiert sind und deshalb vom Betriebssystem abhängig sind.



Für UNIX-Betriebssysteme sind sie jedoch durch den POSIX-Standard standardisiert. Die Low-Level-Funktionen haben eine große Bedeutung für die Systemprogrammierung unter UNIX, beispielsweise für die Programmierung der Interprozesskommunikation zwischen Betriebssystemprozessen. Die High-Level-Dateizugriffsfunktionen hingegen verstecken das Betriebssystem. Damit sind Programme, welche nur die High-Level-Dateizugriffsfunktionen verwenden, portabel, sprich sie können auf andere Rechner beziehungsweise Betriebssysteme übertragen werden und laufen auch dort.

### 16.4.3 Weiterführende Dokumentation der Funktionen

Sowohl Low-Level- wie auch High-Level-Funktionen bieten eine unglaubliche Vielzahl an Optionen, welche den Rahmen dieses Buches sprengen würden. In den folgenden Kapiteln werden nur die wichtigsten Vertreter der High-Level-Funktionen mit den geläufigsten Parametern beschrieben.

Für eine umfassende Dokumentation dieser Funktionen wird auf die in jeder heutigen IDE eingebaute Hilfe verwiesen. Unter Unix oder Systemen mit Unix-ähnlichen Konsolen können alle benötigten Informationen beispielsweise auch mittels folgender Konsolenanweisung abgefragt werden:

```
man 3 printf
```

Diese sogenannten man pages (Manual-Seiten) sind in sämtlichen Unix-ähnlichen Systemen eingebaut. Auch im Web können mittels einfacher Suche nach „man printf“ oder einfach „printf“ eben diese man pages abgerufen werden.

## 16.5 Der File-Pointer bei High-Level-Funktionen

Dieses Unterkapitel behandelt den allgemeinen Umgang mit File-Pointern. Aufgrund der Vielzahl an Möglichkeiten, welche die Standardbibliothek `stdio.h` bietet, können hier nur die wichtigsten Funktionen besprochen werden. Allesamt werden sie am Ende in Kapitel [→ 16.5.6](#) in einem Beispiel angewendet.

Der Zugriff auf eine Datei über die High-Level Dateizugriffsfunktionen wird über einen File-Pointer durchgeführt. Der File-Pointer wird beim Öffnen einer Datei von der Funktion `fopen()` zurückgegeben und zeigt auf eine Struktur vom Typ **FILE**. Je nach Betriebssystem und Compiler sind die Inhalte der **FILE**-Struktur unterschiedlich.



Variablen vom Typ **FILE\*** bezeichnet man im Allgemeinen als **Stream**. Andere Bezeichnungen sind **File-Pointer** oder **Dateivariablen**. Die Struktur **FILE** ist in der Bibliothek `<stdio.h>` definiert.



Die genaue Struktur von FILE muss nicht bekannt sein, da der Inhalt der Struktur nur für die Implementierung der High-Level-Dateifunktionen relevant ist.

Die Struktur FILE verfügt über einen **Dateipuffer**. Dies bedeutet, dass die Funktionen des High-Level-Dateizugriffs zum Lesen und Schreiben grundsätzlich nur auf einen Dateipuffer zugreifen, der sich im Hauptspeicher befindet. Dies beschleunigt Lese- und Schreibprozesse erheblich, da Zugriffe auf den Hauptspeicher um ein Vielfaches schneller sind als auf eine Datei. Das tatsächliche Schreiben und Lesen des Puffers wird nur dann durchgeführt, wenn dies erforderlich ist. Bei Ausgabedateien kann man mittels der `fflush()`-Funktion den Puffer manuell leeren (siehe Kapitel [→ 16.5.3](#)).

Die Struktur FILE speichert zudem Flags, die den Status einer Datei darstellen. Unter anderem wird der Dateiende-Status (End-Of-File, EOF) und der Fehlerstatus des Streams vermerkt. Diese beiden Status können mit den Funktionen `feof()` und `ferror()` abgefragt werden (siehe Kapitel [→ 16.5.5](#)).

### 16.5.1 Standard-File-Pointer

Es gibt bereits vordefinierte Standard-File-Pointer, die das Ansprechen der Ein- und Ausgabegeräte gestatten. Die folgenden File-Pointer sind in `<stdio.h>` als symbolische Konstanten definiert:

| File-Pointer        | Ansprechbare Ein-/Ausgabegeräte    |
|---------------------|------------------------------------|
| <code>stdin</code>  | Standardeingabe (Tastatur)         |
| <code>stdout</code> | Standardausgabe (Bildschirm)       |
| <code>stderr</code> | Standardfehlerausgabe (Bildschirm) |

Diese File-Pointer entsprechen konstanten Pointern auf den Typ `FILE*`.

Unter dem Betriebssystem UNIX gibt es die drei reservierten Namen für die Standardkanäle **`stdin`**, **`stdout`** und **`stderr`**. Beim Start eines Programms werden die File-Pointer automatisch vom Laufzeitsystem mit den entsprechenden Standardkanälen und damit mit den in der obigen Tabelle genannten Ein-/Ausgabegeräten verknüpft. Die File-Pointer sind damit initialisiert – ein Aufruf von `fopen()` ist nicht nötig und auch nicht möglich.

Auf der Ebene des Betriebssystems kann die Zuordnung der Standardkanäle zu Geräten beziehungsweise Dateien aber auch geändert werden – etwa durch Umlenkung oder Pipelining (siehe dazu Kapitel [→ 16.3](#)). Die Standardfehlerausgabe ist eine zusätzliche Einheit zu stdout. Schreibt man Fehlermeldungen nicht nach stdout, sondern nach stderr, so kann verhindert werden, dass die Fehlerausgaben beim Umlenken der Standardausgabe nicht mehr am Bildschirm sichtbar sind.

Man sollte es sich zur Angewohnheit machen, Fehler und Warnungen nie mit `printf(...)` oder mit `fprintf(stdout, ...)` nach stdout auszugeben. Man sollte sie immer nach stderr mit `fprintf(stderr, ...)` ausgeben.



Damit erreicht man nämlich, dass die Fehler- und Warnungsausgaben am Bildschirm sichtbar bleiben und man sehr leicht feststellen kann, ob und wie oft es Fehler und Warnungen gab.

### 16.5.2 Öffnen und Schließen von Dateien

Wenn mit Dateien gearbeitet wird, müssen sie normalerweise zuerst geöffnet werden. Dies geschieht beispielsweise, indem angegeben wird, unter welchem Dateinamen eine Datei im Betriebssystem aufzufinden ist. Wenn die Datei erfolgreich geöffnet wurde, kann man andere Funktionen aufrufen, um die Datei zu manipulieren oder um Daten aus ihr herauszulesen. Am Ende der Verarbeitung wird die Datei wieder geschlossen.

Die Funktion `fopen()` öffnet eine Datei mit bestimmten Zugriffsrechten wie beispielsweise zum Lesen oder Schreiben. Die Funktion `fclose()` schließt alle Schreibvorgänge ab und schließt die Datei.



Die beiden Funktionen `fopen()` und `fclose()` haben die folgenden Prototypen:

```
FILE* fopen (const char* filename, const char* mode);  
int    fclose(FILE* stream);
```



In C11 sind die beiden Parameter von `fopen()` zusätzlich mit dem `restrict`-Schlüsselwort deklariert.

Die Funktion `fopen()` öffnet die durch `filename` definierte Datei mit dem im Parameter `mode` angegebenen Schalter für das Zugriffsrecht. Wenn die Datei erfolgreich geöffnet werden konnte, gibt die Funktion einen Pointer auf `FILE` zurück. Im Fehlerfall wird `NULL` zurückgegeben.

Wenn die Datei im Programm nicht mehr benötigt wird, kann einfach die Funktion `fclose()` mit dem entsprechenden File-Pointer aufgerufen werden. Dadurch werden sämtliche Schreibvorgänge abgeschlossen, wenn noch welche abzuarbeiten sind. Sprich, der Puffer wird in die Datei geschrieben und der Datei-Stream wird vom Programm getrennt.

Grundsätzlich sollte eine Datei immer sofort nach Abschluss der Dateibearbeitung geschlossen werden. Dies verhindert einen eventuellen Datenverlust bei einem späteren Programmabsturz. Außerdem ist die Anzahl der gleichzeitig geöffneten Dateien durch das Betriebssystem begrenzt.



Ein einfaches Beispiel:

```
FILE* fp = fopen ("antwort.txt", "w");
if (fp != NULL) {
    fprintf(fp, "Zweiundvierzig\n");
    fclose(fp);
}
```

Hier wird eine Datei zum Schreiben im Textmodus geöffnet. Der Parameter `mode` wird hier mittels `"w"` angegeben. Das `w` ist dabei ein sogenannter „Schalter“. Folgendes sind die wichtigsten Schalter:

| Schalter | Bedeutung                                                                                                                                            |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| r        | Öffnen einer Datei ausschließlich zum Lesen.                                                                                                         |
| w        | Datei zum Schreiben erzeugen. Existiert die Datei bereits, so wird sie überschrieben.                                                                |
| a        | Öffnen einer Datei zum Anfügen. Existiert die Datei bereits, so werden neue Daten an das Dateieinde angefügt, ansonsten wird die Datei neu erstellt. |
| b        | Zusatz-Schalter für das Öffnen einer Datei im Binärmodus.                                                                                            |

Die Schalter können teilweise auch kombiniert werden. So ist es mit dem Modus "rw" beispielsweise möglich, eine Datei sowohl zu lesen als auch zu schreiben. Auch kann man sich ab dem C11-Standard gar vom Betriebssystem Exklusiv-Rechte erbitten. Hier wird auf die weiterführende Dokumentation verwiesen.

Zum Öffnen einer Datei für eine binäre Ein-/Ausgabe muss zusätzlich der Buchstabe b als Schalter angegeben werden, also "rb" oder "wb". Wenn dieser Schalter nicht angegeben wird, so wird eine Datei im Textmodus geöffnet.

Der Unterschied zwischen einer Datei im **Text-** und im **Binärmodus** ist, dass im Textmodus – im Falle von Windows-Compilern – beim Schreiben eines Zeilenendes ein LF durch ein CR LF (Carriage Return, Line Feed) ersetzt wird, zum anderen wird beim Lesen ein CR LF durch ein LF ersetzt. Wenn die Datei im Binärmodus geöffnet wurde, wird dieser Ersetzungsvorgang nicht durchgeführt.



Bei UNIX-Compilern wird der Schalter b zwar akzeptiert, es gibt jedoch keinen Unterschied zwischen Text- und Binärmodus. Es wird auf den Ersetzungsvorgang am Zeilenende komplett verzichtet.

Zusätzlich zu `fopen()` und `fclose()` gibt es die Funktion `freopen()`, welche versucht, einen bereits geöffneten File-Pointer zu schließen und danach mittels eines neuen Modus erneut zu öffnen. Hier wird auf eine genauere Behandlung verzichtet.

### 16.5.3 Puffersteuerung

Wie bereits besprochen, hat ein File-Pointer einen **Dateipuffer**, welcher einzelne Lese- und Schreib-Operationen enorm beschleunigen kann. Wünscht man an einem Punkt im Programm explizit, dass der Puffer geleert werden soll, so kann dies mit einem Aufruf der Funktion `fflush()` erreicht werden. Mit den Funktionen `setbuf()` und `setvbuf()` kann zudem für jeden File-Pointer einzeln gesteuert werden, was genau gepuffert werden soll.

Die Prototypen für die Funktionen sind die folgenden:

```
int  fflush (FILE* stream);  
int  setvbuf(FILE* stream, char* buf, int mode, size_t size);  
void setbuf (FILE* stream, char* buf);
```

In C11 sind die ersten beiden Parameter von `setvbuf()` und `setbuf()` zusätzlich mit dem `restrict`-Schlüsselwort deklariert.

Die Funktion `fflush()` sorgt bei einem Ausgabestrom dafür, dass alle in den Dateipuffern existierenden und noch nicht geschriebenen Daten auf die Festplatte geschrieben werden. Die Wirkungsweise von `fflush()` auf einen Eingabestrom ist laut Standard undefiniert.



Mittels der Funktion `setvbuf()` kann gesteuert werden, wie groß der Puffer eines File-Pointers sein soll, wie er genutzt werden soll, und es kann sogar ein eigener Puffer angegeben werden. Die Funktion `setbuf()` ist nur eine vereinfachte Version von `setvbuf()` und ruft selbige Funktion mit vorgegebenen Standardwerten auf. Hier wird auf die weiterführende Dokumentation verwiesen.

Ein Aufruf der Funktion `fclose()` führt automatisch auch einen Aufruf von `fflush()` aus. So werden alle noch nicht geschriebenen Daten in die Datei gespeichert.

### 16.5.4 Navigieren innerhalb von Dateien

Ein File-Pointer speichert intern, wo genau sich das Programm momentan in der Datei befindet. Wird eine Datei beispielsweise im Lesemodus "r" geöffnet, so zeigt der interne **Dateipositionszeiger** zu Beginn auf den Anfang der Datei. Beim Lesen einer Datei rückt der Dateipositionszeiger allmählich vor, bis schlussendlich das Ende der Datei erreicht wird. Wenn gewünscht, kann jedoch der Dateipositionszeiger auch beliebig verschoben werden.

Mittels der Funktion `ftell()` kann ermittelt werden, an welcher Position sich der Dateipositionszeiger gerade befindet. Mittels `fseek()` kann der Zeiger auf eine beliebige Position innerhalb der Datei gesetzt werden. Die Funktion `rewind()` setzt den Zeiger zurück zum Anfang der Datei.



Folgendes sind die Prototypen der Funktionen:

```
long ftell (FILE* stream);  
int  fseek (FILE* stream, long offset, int whence);  
void rewind(FILE* stream);
```

Diese Funktionen erlauben es, den Dateipositionszeiger innerhalb von binären Dateien beliebig zu setzen. Für Textdateien jedoch funktionieren diese Funktionen nur unzuverlässig, da die automatische Umwandlung von CR in CR LF unter Windows ein einwandfreies Arbeiten bei Dateien im Textmodus verhindert. Für Textdateien, welche Zeichen des Typs `wchar_t` speichern, gibt es seit C11 zusätzlich die Funktionen `fgetpos()` und `fsetpos()`, welche eine exakte Positionierung erlauben. Hier in diesem Buch werden sie nicht weiter angesprochen.

Die Funktion `fseek()` setzt den Dateipositions-Zeiger der durch `stream` definierten Datei auf die Position, die `offset` Bytes von `whence` entfernt ist. Für den Parameter `whence` sind in `<stdio.h>` 3 Konstanten definiert:

|                       |                                           |
|-----------------------|-------------------------------------------|
| <code>SEEK_SET</code> | offset ist relativ zum Dateianfang        |
| <code>SEEK_CUR</code> | offset ist relativ zur aktuellen Position |
| <code>SEEK_END</code> | offset ist relativ zum Dateende           |

Die Funktion `rewind()` positioniert den Dateipositions-Zeiger der durch `stream` definierten Datei an den Dateianfang und setzt zusätzlich sämtliche Fehler-Flags zurück.

### 16.5.5 Fehlerbehandlung

Ein File-Pointer speichert nebst dem obengenannten Dateipositionszeiger auch **Status-** und **Fehler-Flags**. Beispielsweise ist es beim Lesen einer Datei wichtig zu wissen, ob der Dateipositionszeiger am Ende der Datei angelangt ist.

In einigen Funktionen zur Ein-/Ausgabe in Dateien kann nicht zwischen einem eigentlichen Fehler bei der Dateiverarbeitung und dem Erreichen des Dateiendes unterschieden werden. Um den Stream in einem solchen Fall genauer zu analysieren, werden folgende Funktionen bereitgestellt:

```
int feof (FILE* stream);  
int ferror (FILE* stream);  
void clearerr(FILE* stream);
```

Die Funktion `feof()` prüft, ob das Dateiende erreicht ist. Die Funktion `ferror()` prüft das Fehlerflag einer Datei. Die Funktion `clearerr()` setzt das Dateiende- und das Fehlerflag einer Datei zurück.



Die Funktionen `feof()` und `ferror()` geben eine Zahl ungleich null zurück, wenn das entsprechende Flag gesetzt ist. Was genau die zurückgegebene Zahl aussagt, ist jedoch implementationsspezifisch.

Bei jeder Funktion der Standardbibliotheken (nicht nur bei Verwendung von `<stdio.h>`), bei welcher ein Fehler passiert, wird die globale Fehler-Variable `errno` mit einem fest vorgegebenen Wert gesetzt. Alle möglichen Fehlernummern können in der Standard-Headerdatei `<errno.h>` nachgelesen werden. Zudem kann man mittels der Funktion `perror()` sich eine lesbare Fehlermeldung ausgeben lassen.

```
int perror(const char* s);
```

Der übergebene String wird mit einem Doppelpunkt ausgegeben, gefolgt von der entsprechenden Fehlermeldung.

### 16.5.6 Beispiel für den Umgang mit File-Pointern

Hier wird in einem etwas ausführlicheren Beispiel gezeigt, wie mit den in diesem Kapitel vorgestellten Funktionen umgegangen wird. Es wird eine Datei eingelesen mit einzelnen Zahlen in mehreren Zeilen. Aus dieser Datei wird eine Ausgabedatei generiert, in welcher die Zahlen als auf 1 normierte, kumulative Verteilfunktion aufgelistet sind:

verteilung.c

```
#include "stdlib.h"
#include "stdio.h"

int main(void) {
    char number[10];
    int sum = 0;
    int cumulation = 0;

    // Oeffnen der Input-Datei
    FILE* infile = fopen("input.txt", "r");
    if (infile == NULL) {
        perror("Inputdatei"); exit(1);
    }

    // Oeffnen der Output-Datei
    FILE* outfile = fopen("result.txt", "w");
    if (outfile == NULL) {
        perror("Outputdatei"); exit(1);
    }

    // Lesen der Zahlen:
    while (!feof(infile)) {
        fgets(number, 10, infile);    // Einlesen einer Zeile
        sum += atoi(number);          // Umwandeln in int
    }

    // Zuruecksetzen der Input-Datei. Auch moeglich: rewind(infile);
    fseek(infile, 0, SEEK_SET);

    // Berechnen der kumulativen Verteilfunktion
    while (!feof(infile)) {
        fgets(number, 10, infile);
        fprintf(outfile, "%f\n", (double)cumulation / sum);
        cumulation += atoi(number);
    }
    fprintf(outfile, "%f\n", 1.);
}
```

```
fflush(outfile); // Nicht noetig. Einfach zur Demo.
printf("Datei result.txt erfolgreich beschrieben.\n");

// Schliessen der Dateien
fclose(outfile);
fclose(infile);

return 0;
}
```

Die Datei `input.txt` auf der linken Seite ergibt die Datei `result.txt` auf der rechten Seite

| <code>input.txt</code> | <code>result.txt</code> |
|------------------------|-------------------------|
| 54                     | 0.000000                |
| 63                     | 0.210117                |
| 88                     | 0.455253                |
| 52                     | 0.797665                |
|                        | 1.000000                |

Die Ausgabe dieses Beispiels könnte nun beispielsweise als Eingabe für ein Programm dienen, welches aus diesen Zahlen eine grafische Darstellung dieser kumulativen Verteilfunktion erstellt.

### 16.5.7 Operationen auf Dateien

Der C-Standard sieht ein paar wenige Funktionen vor, mit denen man mit dem Betriebssystem kommunizieren kann, um neue Dateien anzulegen, Dateien umzubenennen oder gar Dateien zu löschen.

Dies sind die Prototypen der Funktionen:

```
char* tmpnam (char* s);
FILE* tmpfile(void);
int rename (const char* old, const char* new);
int remove (const char* filename);
```

Die Funktion `tmpnam()` erstellt einen eindeutigen Dateinamen, welcher keinem anderen Namen einer existierenden Datei entspricht. Mit der Funktion `tmpfile()` erstellt und öffnet das Betriebssystem eine neue, binäre Datei mit Schreibzugriff, welche einen eindeutigen Namen besitzt, aber nur temporär existiert. Die Datei wird am Ende des Programmes oder beim Schließen der Datei automatisch wieder gelöscht.



Mit der Funktion `rename()` kann eine Datei umbenannt werden. Die Funktion `remove()` löscht eine Datei.



## 16.6 Einfaches Schreiben in Dateien mit High-Level-Funktionen

Wenn eine Datei erfolgreich im Schreib-Modus geöffnet wurde, kann sie mit Hilfe verschiedener Funktionsaufrufe mit Daten gefüllt werden. Das Schreiben von Daten geschieht grundsätzlich zeichenweise, das heißt, jedes Byte wird einzeln vom Hauptspeicher in die Datei geschrieben.

Die Funktion `fputc()` ist die grundlegendste Funktion, um Daten in eine Datei zu schreiben. Unter dem Namen `putc()` ohne `f` existiert zudem ein Makro, welches exakt dasselbe tut wie `fputc()`, jedoch als Makro ausprogrammiert ist. Das Makro sollte somit nicht verwendet werden, wenn dessen Parameter Ausdrücke mit Nebeneffekten darstellen. Die Prototypen von `fputc()` beziehungsweise `putc()` sind:

```
int fputc(int c, FILE* stream);  
int putc (int c, FILE* stream);
```

Die Funktion `fputc()` schreibt das (von `int` in `unsigned char` umgewandelte) Zeichen `c` in die durch `stream` definierte Datei. Die Funktion `fputc()` liefert bei fehlerfreier Abarbeitung das geschriebene Zeichen `c` zurück. Im Fehlerfall wird das Fehlerflag für den Stream gesetzt und `EOF` zurückgegeben.



Die Funktion `fputc()` schreibt ein einzelnes Zeichen (ein Byte) in einen gegebenen File-Pointer. Wenn die Datei als Binärdatei geöffnet wurde, so wird das Byte ohne weitere Konvertierung direkt in die Datei geschrieben. Wenn die Datei als Textdatei geöffnet wurde, so werden gegebenenfalls Zeilenende-Zeichen je nach Betriebssystem umkonvertiert.



Alle anderen Schreib-Funktionen bauen auf `fputc()` auf. Man muss sich also beim Öffnen der Datei genau überlegen, ob die Datei als Binärdatei oder als Textdatei behandelt werden soll.

Im Folgenden werden die erweiterten Schreib-Funktionen der Standardbibliothek `<stdio.h>` vorgestellt. Hier wird grob unterschieden zwischen Objekten und formatierten Strings.

### 16.6.1 Schreiben von Objekten und Strings

Unter Objekten werden hier beliebig große Strukturen verstanden, welche als ein Array von Bytes angesprochen werden. Die folgenden Funktionen erlauben es, ganze Arrays von solchen Objekten und Strings mittels eines einzigen Funktionsaufrufes in eine Datei zu speichern:

```
size_t fwrite(  
    const void* ptr,  
    size_t size,  
    size_t nmemb,  
    FILE* stream);  
  
int fputs(  
    const char* s,  
    FILE* stream);
```

Die Funktion `fwrite()` schreibt ganze Arrays von Objekten in eine Datei. Die Funktion `fputs()` schreibt einen String in eine Datei.



Die Funktion `fwrite()` schreibt `nmemb` Objekte der Größe `size` aus dem Array, auf das der Pointer `ptr` zeigt, in die durch `stream` definierte Datei. Insgesamt werden maximal `nmemb * size` Bytes geschrieben.

Die Funktion `fputs()` schreibt den String, auf den der Pointer `s` zeigt, in die durch `stream` definierte Datei. Beim Schreiben wird das Stringende-Zeichen `'\0'` nicht in die Datei geschrieben.

Der Datentyp `size_t` ist in `<stddef.h>` definiert und wurde bereits im Zusammenhang mit dem `sizeof`-Operator in Kapitel [9.9.1](#) erläutert.

### 16.6.2 Schreiben von ganzen Strings auf die Standardausgabe

Für die Ausgabe von ganzen Strings ohne Formatierung auf dem Standard-Ausgabegerät gibt es folgende Funktions-Prototypen:

```
int puts (const char* s);  
int putchar(const char* s);
```

Hierbei entspricht `puts()` der Funktion `fputs()` und `putchar()` entspricht dem Makro `putc()`. Bei beiden Funktionen wird als `stream` die Standardausgabe `stdout` verwendet.

Bei der Funktion `puts()` wird die Ausgabe mit einem automatisch angehängten Zeilenende ergänzt. Das abschließende Nullzeichen `'\0'` wird jedoch nicht geschrieben.

## 16.7 Schreiben von formatierten Strings

Die Funktion **printf()** („print formatted“) dient zur formatierten Ausgabe von Werten von Ausdrücken auf der Standardausgabe.



Der Funktions-Prototyp von `printf()` lautet:

```
int printf(const char* format, ...);
```

In C11 ist der Parameter `format` zusätzlich mit dem `restrict`-Schlüsselwort deklariert. Der Prototyp steht in der Bibliothek `<stdio.h>`.

### 16.7.1 Varianten von `printf()`

Mit `printf()` werden Werte formatiert ausgegeben. Ein Ausdruck mit einem Wert kann all das sein, was einen Wert hat, beispielsweise eine Konstante, eine Variable, ein Funktionsaufruf, der einen Wert zurückgibt, oder die Verknüpfung eines Ausdrucks mit einem anderen Ausdruck durch Operatoren.

Bevor jedoch erklärt wird, wie genau Ausdrücke formatiert werden können, werden die verschiedenen Varianten von `printf()` vorgestellt.

Von der Funktion `printf()` gibt es verschiedene Implementierungen, welche allesamt dieselben Formatierungen vornehmen, jedoch unterschiedliche Ein- und Ausgänge besitzen. Die entsprechenden Funktionen werden durch unterschiedliche Präfixe vor dem `printf` ausgedrückt:

|                        |                                                                                   |
|------------------------|-----------------------------------------------------------------------------------|
| <code>printf</code>    | Ausgabe nach <code>stdout</code>                                                  |
| <code>fprintf</code>   | Ausgabe in Datei ( <code>f</code> = File)                                         |
| <code>sprintf</code>   | Ausgabe in String                                                                 |
| <code>snprintf</code>  | Ausgabe in String mit Maximallänge                                                |
| <code>vprintf</code>   | Ausgabe nach <code>stdout</code> , Parameter aus variabler Parameterliste         |
| <code>vfprintf</code>  | Ausgabe in Datei ( <code>f</code> = File), Parameter aus variabler Parameterliste |
| <code>vsprintf</code>  | Ausgabe in String, Parameter aus variabler Parameterliste                         |
| <code>vsnprintf</code> | Ausgabe in String mit Maximallänge, Parameter aus variabler Parameterliste        |

Die Funktion `printf()` muss in der Lage sein, beliebig viele Argumente zu formatieren. Daher enthält der Prototyp von `printf()` auch die Auslassung `(...)`. Die Auslassung wird meist als Ellipse bezeichnet. Wenn die Funktion jedoch das `v`-Präfix besitzt, so werden die Eingabewerte anstelle der Auslassung mittels einer variablen Parameterliste mit dem Typ `va_list` angegeben. Diese Funktionen werden hier nicht weiter behandelt. Weitere Informationen zu variablen Parameterlisten und den Ellipsen können in Kapitel [→ 11.7](#) nachgelesen werden.

Das Präfix `f` sorgt dafür, dass die Ausgabe nicht auf die Standardausgabe `stdout` erfolgt, sondern in einen beliebigen File-Pointer. Der Prototyp der Funktion `fprintf()` lautet:

```
int fprintf(FILE* stream, const char* format, ...);
```

In C11 sind die beiden Parameter zusätzlich mit dem `restrict`-Schlüsselwort deklariert.

Die Präfixe `s` und `sn` sorgen dafür, dass die Ausgabe nicht auf die Standardausgabe `stdout` erfolgt, sondern in einen String. Folgendes sind die Prototypen:

```
int sprintf (char* s, const char* format, ...);  
int snprintf(char* s, size_t n, const char* format, ...);
```

In C11 sind die beiden Pointer-Parameter wiederum zusätzlich mit dem `restrict`-Schlüsselwort deklariert.

Bei beiden Funktionen wird die formatierte Ausgabe in den durch `s` gegebenen String gespeichert. Beide Funktionen hängen automatisch an das Ende des formatierten Strings ein Null-Byte an. Man muss selbst dafür sorgen, dass der in `s` gegebene Puffer groß genug ist für alle Bytes inklusive des abschließenden Nullzeichens.

Bei der `sn`-Variante kann man jedoch explizit angeben, wie groß der String maximal sein darf. Die Funktion wird sodann die Ausgabe rechtzeitig abbrechen, damit der Puffer nicht überlaufen kann.

### 16.7.2 Formatieren von Werten

Der Prototyp der Funktion `printf()` deklariert vor der Ellipse nur einen einzigen formalen Parameter, den Parameter `format`. Dieser Parameter ist vom Typ Pointer auf `char` und stellt den sogenannten **Format-String** dar. Mit diesem String wird gesteuert, wie die restlichen Parameter formatiert werden sollen.

Der Format-String enthält sogenannte „**Formatelemente**“, welche allesamt mit dem Prozent-Zeichen `%` eingeleitet werden. Für jedes Formatelement in dem String des ersten Argumentes muss ein weiteres Argument als Teil der Ellipse übergeben werden. An derjenigen Stelle des Strings, an der das Formatelement steht, erfolgt die Ausgabe des entsprechenden Argumentes.

```
printf("Einnahmen an Tag %d: %d Euro", tagNummer, einnahmen);
```

Ergibt beispielsweise:

```
Einnahmen an Tag 7: 3558 Euro
```

Die Reihenfolge der weiteren Argumente der Funktion `printf()` und deren Typ muss mit der Reihenfolge und Art der Formatelemente im Format-String übereinstimmen. Der Format-String wird von links nach rechts abgearbeitet. Gewöhnliche Zeichen in diesem String werden auf die Standardausgabe geschrieben. Die Formatelemente bestimmen das Format der auszugebenden weiteren Argumente.



Formatelemente beginnen mit einem %-Zeichen. Jedes Formatelement wird durch Umwandlungszeichen beendet.



Ein Umwandlungszeichen definiert, was der übergebene Parameter für einen Typ besitzt und wie er dargestellt werden soll. Folgende Tabelle definiert die wichtigsten Umwandlungszeichen:

| <b>Ausgabe von Integer</b>           |              |                                                                                                                                                            |
|--------------------------------------|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| d, i                                 | int          | dezimal, gegebenenfalls mit Vorzeichen.                                                                                                                    |
| o                                    | unsigned int | oktal ohne Vorzeichen (ohne führende null).                                                                                                                |
| x, X                                 | unsigned int | hexadezimal ohne Vorzeichen in Klein- beziehungsweise Großbuchstaben (ohne führendes 0x beziehungsweise 0X).                                               |
| u                                    | unsigned int | ohne Vorzeichen in dezimaler Form.                                                                                                                         |
| c                                    | int          | als Zeichen. Dabei wird das Argument in den Typ unsigned char gewandelt.                                                                                   |
| b                                    | int          | binär, existiert erst ab dem Standard C23.                                                                                                                 |
| <b>Ausgabe von Strings</b>           |              |                                                                                                                                                            |
| s                                    | char*        | als String. Zeichen des Arrays werden bis zum Nullzeichen (jedoch nicht einschließlich) geschrieben.                                                       |
| <b>Ausgabe von Gleitpunkt-Zahlen</b> |              |                                                                                                                                                            |
| f                                    | double       | als dezimale Zahl.                                                                                                                                         |
| e, E                                 | double       | als Exponentialzahl, wobei das den Exponenten anzeigende e klein beziehungsweise groß geschrieben ist.                                                     |
| g, G                                 | double       | als Exponentialzahl beziehungsweise als Dezimalzahl in Abhängigkeit vom Wert. Nullen am Schluss sowie ein Dezimalpunkt am Schluss werden nicht ausgegeben. |
| <b>Weiteres</b>                      |              |                                                                                                                                                            |
| p                                    | Pointer-Typ  | als Adresse.                                                                                                                                               |
| %                                    |              | als %-Zeichen.                                                                                                                                             |
| n                                    | int*         | In das übergebene Argument wird die Zahl der von printf() geschriebenen Zeichen abgelegt.                                                                  |

Wenn die Formatelemente nicht zum Typ der Argumente passen oder die Zahl der Argumente nicht stimmt, wird die Ausgabe von `printf()` falsch oder kann gar zum Absturz des Programmes führen.



Enthält der Format-String keine Formatelemente, so stellt dieser String einfach einen String dar, der ausgegeben werden soll, wie im Beispiel:

```
printf("Hier gibt es keine Formatelemente.");
```

Je nach Formatelement können zwischen dem einleitenden %-Zeichen und dem abschließenden Umwandlungszeichen weitere Formatierungs-Zeichen stehen. Diese werden in den folgenden Unterkapiteln genauer erläutert.

### 16.7.3 Wichtige Formate für die Ausgabe von Integer

Bei Integer wird die Feldbreite der Ausgabe angegeben durch `%nd`. Die Zahl `n` legt dabei die Feldbreite, also die Anzahl der Stellen, fest.



Ist die Zahl schmaler als die angegebene Feldbreite, so wird mit Leerzeichen bis zur angegebenen Feldbreite aufgefüllt. Da die angegebene Feldbreite vom Compiler ignoriert wird, wenn sie nicht ausreicht, spricht man oft von der sogenannten minimalen Feldbreite. Es ist zu beachten, dass ein eventuelles Vorzeichen ebenfalls in diese Feldbreite miteinbezogen wird. Bei negativen Zahlen hat also stets eine Ziffer weniger Platz.

Wenn vor der Feldbreite eine `0` (null) geschrieben wird, so wird die Zahl von links her mit Nullen aufgefüllt, sodass sie die Feldbreite voll ausnutzt. Wird vor die Feldbreite ein Minus-Zeichen geschrieben, so wird die Zahl nicht rechtsbündig, sondern linksbündig ausgegeben. Wenn zusätzlich ein Plus-Zeichen geschrieben wird, wird das Vorzeichen der Zahl auch für positive Werte ausgegeben.

Im folgenden Beispiel sind die wichtigsten Formatelemente zu beobachten:

```
int x = 99;
printf("%d Dalmatiner\n", x);
printf("%5d Dalmatiner\n", x);
printf("%05d Dalmatiner\n", x);
printf("%-5d Dalmatiner\n", x);
printf("%+5d Dalmatiner\n", x);
printf("%+05d Dalmatiner\n", x);
printf("%-+5d Dalmatiner\n", x);
```

Die Ausgabe ist:

```
99 Dalmatiner
   99 Dalmatiner
00099 Dalmatiner
99  Dalmatiner
+99 Dalmatiner
+0099 Dalmatiner
+99  Dalmatiner
```

#### 16.7.4 Wichtige Formate für die Ausgabe von Gleitpunkt-Zahlen

Im Folgenden werden die verschiedenen Formate von Gleitpunkt-Zahlen betrachtet:

```
double a = 0.000006;
double b = 123.4;
printf("%e", a);
printf("%E", a);
printf("%f", a);
printf("%g", a);
printf("%G", a);
printf("%g", b);
printf("%G", b);
```



Die Ausgabe ist:

```
a:
6.000000e-06
6.000000E-06
0.000006
6e-06
6E-06
b:
123.4
123.4
```

Die Formatelemente %e und %E dienen zur Darstellung einer Gleitpunkt-Zahl als Exponentialzahl mit Signifikante und Exponent.



Dabei wird das e, das den Exponenten charakterisiert, bei Angabe von %e als kleines e ausgegeben, bei Angabe von %E als großes E. Das Formatelement %f dient zur Ausgabe als Dezimalzahl. %g und %G geben je nach Größe der Zahl diese als Exponentialzahl oder als Dezimalzahl aus.

Mit Hilfe der Formatelemente von printf() zur Ausgabe von Gleitpunkt-Zahlen als Dezimalzahlen kann die Gesamtbreite der Ausgabe (Feldbreite) und die Zahl der Stellen hinter dem Dezimalpunkt beeinflusst werden, wie aus den folgenden Beispielen ersichtlich ist:

|       |                                                                                                                                       |
|-------|---------------------------------------------------------------------------------------------------------------------------------------|
| %f    | Gibt eine Gleitpunkt-Zahl im Default-Format des Compilers aus.                                                                        |
| %5.2f | Gibt eine Gleitpunkt-Zahl mit insgesamt 5 Stellen aus, 2 hinter dem Punkt, 1 für den Punkt und 2 vor dem Punkt.                       |
| %5.0f | Die Gleitpunkt-Zahl wird mit 5 Stellen ausgegeben ohne Punkt und ohne Stelle nach dem Punkt.                                          |
| %.3f  | Es sollen 3 Stellen hinter dem Punkt und der Punkt ausgegeben werden. Für die Zahl der Stellen vor dem Punkt erfolgt keine Anweisung. |

Für die Formatangaben vor dem Dezimalpunkt können dieselben Formatierungsmöglichkeiten benutzt werden wie bei Integer.

Die folgenden Zeilen verwenden diese Formatelemente:

```
float zahl = 12.3456f;  
printf("zahl = %f\n", zahl);  
printf("zahl = %5.2f\n", zahl);  
printf("zahl = %5.0f\n", zahl);  
printf("zahl = %.3f\n", zahl);
```

Dies erzeugt folgende Ausgabe:

```
zahl = 12.345600  
zahl = 12.35  
zahl = 12  
zahl = 12.346
```

Es ist dabei zu beachten, dass die Zahl gegebenenfalls automatisch auf die entsprechende Anzahl Stellen gerundet wird.

### 16.7.5 Wichtige Formate für die Ausgabe von Strings

Strings werden ausgegeben mit dem Formatelement %s.



Dies zeigen folgende Beispiele:

```
const char* s1 = "ist";  
printf("Programmieren %s einfach\n", s1);  
printf("Programmieren %6s einfach\n", s1);  
printf("Programmieren %-6s einfach\n", s1);
```

Die Ausgabe ist:

```
Programmieren ist einfach  
Programmieren  ist einfach  
Programmieren ist  einfach
```

Bei der Ausgabe von char-Arrays erhält die Funktion `printf()` einen Pointer auf das erste Zeichen eines char-Arrays. Die Funktion `printf()` gibt die Zeichen des Arrays aus, bis ein Nullzeichen `'\0'` gefunden wird. Das Nullzeichen wird nicht ausgegeben.



Strings mit Angabe einer positiven Feldbreite werden rechtsbündig ausgegeben. Wird die Feldbreite negativ angegeben, so wird der String linksbündig ausgegeben.



## 16.8 Lesen von Dateien mit High-Level-Funktionen

Für das Lesen von Dateien existieren grundsätzlich äquivalente Funktionen wie für das Schreiben von Dateien (gemäß Kapitel [→ 16.6](#)). Beim Lesen werden die Funktionen jedoch mit `fget()` anstatt mit `fput()`, mit `fread()` anstatt mit `fwrite()` beziehungsweise mit `fscan()` anstatt mit `fprint()` bezeichnet.

Grundsätzlich können die obigen Erklärungen somit analog für das Lesen von Dateien angewendet werden. Dennoch gibt es bei den Lese-Funktionen ein paar nennenswerte Ausnahmen. Im Folgenden werden die Lese-Funktionen der Standardbibliothek mit demselben Aufbau wie bei den Schreib-Funktionen durchgearbeitet.

Wenn eine Datei erfolgreich im Lese-Modus geöffnet wurde, können die in der Datei enthaltenen Daten mit Hilfe verschiedener Funktionsaufrufe in Variablen des Programmes eingelesen werden. Genauso wie beim Schreiben von Dateien findet das Lesen grundsätzlich zeichenweise statt. Das heißt, dass jedes Byte einzeln von der Datei gelesen und in den Hauptspeicher übertragen wird.

Die Funktion `fgetc()` ist die grundlegendste Funktion, um Daten von einer Datei einzulesen. Unter dem Namen `getc()` ohne `f` existiert wiederum ein Makro, welches exakt dasselbe tut wie `fgetc()`, jedoch als Makro ausprogrammiert ist. Das Makro sollte nicht verwendet werden, wenn die Parameter Ausdrücke mit Nebeneffekten darstellen. Nebst den folgenden beiden Prototypen gibt es noch die zusätzliche Funktion `ungetc()`, die weiter unten behandelt wird. Hier die Prototypen:

```
int fgetc (FILE* stream);  
int getc (FILE* stream);  
int ungetc(int c, FILE* stream);
```

Die Funktion `fgetc()` liest – wenn vorhanden – das nächste Zeichen als `unsigned char` aus der durch `stream` definierten Datei und konvertiert es nach `int`. Im Fehlerfall wird das Fehlerflag für den Stream gesetzt und EOF zurückgegeben.

Die Funktion `fgetc()` liest ein Zeichen aus einer Datei. Wenn die Datei als Binärdatei geöffnet wurde, so wird das Byte ohne weitere Konvertierung direkt übertragen. Wenn die Datei als Textdatei geöffnet wurde, so werden gegebenenfalls Zeilenende-Zeichen je nach Betriebssystem umkonvertiert.



Alle anderen Lese-Funktionen bauen auf `fgetc()` auf. Man muss sich also beim Öffnen der Datei genau überlegen, ob die Datei als Binärdatei oder als Textdatei behandelt werden soll.

Mit der Funktion `ungetc()` kann man ein Zeichen `c` für eine erneute Leseoperation in die durch `stream` definierte Datei zurückstellen. Dabei verhält sich die Funktion `ungetc()` ähnlich wie die Funktion `fputc()`, wird jedoch durch die Standardbibliotheken dahingehend unterstützt, dass diese Funktion auch auf nur lesbaren Dateien angewendet und mindestens ein Funktionsaufruf garantiert ausgeführt werden kann.

Damit ist es möglich, ganz einfach Parser zu schreiben, welche jeweils ein einzelnes Zeichen lesen und daraufhin entscheiden, ob das Zeichen zusammen mit den nachfolgenden Zeichen als zusammengehöriges Token gelesen werden soll. Das Programm stellt in diesem Falle das Zeichen einfach wieder in die Datei zurück und liest daraufhin das Token vollständig ein.

Die Funktion `ungetc()` stellt ein Zeichen in eine Datei zurück. Direkt nach dem Zurückstellen kann das Zeichen wieder beispielsweise mit der Funktion `fgetc()` gelesen werden.



Im Folgenden werden die erweiterten Lese-Funktionen der Standardbibliothek `<stdio.h>` vorgestellt. Genauso wie bei den Schreib-Funktionen wird grob unterschieden zwischen Objekten und formatierten Strings.

### 16.8.1 Lesen von Objekten und Strings

Die folgenden Funktionen erlauben es, Objekte und Strings mittels eines einzigen Funktionsaufrufes aus einer Datei einzulesen:

```
size_t fread(void* ptr,
             size_t size,
             size_t nmemb,
             FILE* stream);

char* fgets (char* s,
             int n, FILE* stream);
```

Die Funktion `fread()` liest ganze Arrays von Objekten aus einer Datei. Die Funktion `fgets()` liest einen String aus einer Datei.



Die Funktion `fgets()` liest aus der durch `stream` definierten Datei und schreibt das Ergebnis in den Puffer, auf den der Pointer `s` zeigt. Das Lesen wird abgebrochen, wenn entweder das Zeilenende-Zeichen `'\n'` oder das Dateiende erreicht ist oder `n - 1` Zeichen gelesen wurden. Das Zeilenende-Zeichen wird in den Puffer kopiert, ein Dateiende-Zeichen nicht. An die in den Puffer geschriebenen Zeichen wird als Stringende-Zeichen das Zeichen `'\0'` angehängt.

Die Funktion `fread()` liest `nmemb` Objekte der Größe `size` aus der durch `stream` definierten Datei aus und schreibt diese in das Array, auf das der Pointer `ptr` zeigt. Insgesamt werden maximal `nmemb * size` Bytes gelesen.

Für die Eingabe vom Standard-Eingabegerät gibt es zudem folgende Funktions-Prototypen:

```
int    getchar(void);  
char* gets_s (char* s, rsize_t n); // erst seit C11
```

Hierbei entspricht `getchar()` dem Makro `getc()` und `gets_s()` entspricht der Funktion `fgets()`. Beide Funktionen nutzen die Standardeingabe `stdin`.

Bei beiden Funktionen ist zu beachten, dass die Standardeingabe zeilengepuffert ist. Das bedeutet, dass die Funktionen `getchar()` und `gets_s()` solange warten, bis eine Eingabezeile mit <RETURN> abgeschlossen wurde. Erst dann wird der Tastaturpuffer geleert und als Dateipuffer von `stdin` an das Programm übergeben.

Mit der Funktion `getchar()` wird ein einzelnes Zeichen vom Eingabestrom `stdin` gelesen.



Das Zeichen wird als `unsigned char` gelesen und in `int` umgewandelt.

`gets_s()` liest einen String von der Standardeingabe `stdin` in ein Array von `n` Zeichen ein, auf das der Pointer `s` zeigt.



Die Funktion garantiert, dass nicht mehr als `n - 1` Zeichen gelesen werden und der Puffer somit nicht überlaufen kann. Es werden solange Zeichen von `stdin` eingelesen, bis ein Zeilenende-Zeichen oder das Zeichen EOF auftritt. An das letzte Zeichen wird ein Stringende-Zeichen `'\0'` angehängt.

Die Funktion `gets_s()` existiert erst seit dem C11-Standard. Vor dem C11-Standard wurde von den Standardbibliotheken eine Funktion `gets()` unterstützt, welche jedoch keinen Parameter `n` aufwies.

Bei der veralteten Funktion `gets()` konnte ein eingelesener String einen Überlauf des Puffers verursachen und den Speicher beliebig überschreiben.



Sichere Ein- und Ausgabefunktionen werden in Kapitel [→ 16.10](#) behandelt.

## 16.9 Lesen von formatierten Strings

Die Funktion `scanf()` („scan formatted“) dient zum Einlesen von formatierten Werten von der Standardeingabe.



Es sei angemerkt, dass die Standardeingabe zeilengepuffert ist. Das bedeutet, dass Eingaben grundsätzlich durch `<RETURN>` abgeschlossen werden müssen. Der Funktions-Prototyp von `scanf()` lautet:

```
int scanf(const char* format, ...);
```

In C11 ist der Parameter `format` zusätzlich mit dem `restrict`-Schlüsselwort deklariert.

Für `float`-Typen wird bei `scanf()` das Formatelement `%f` verwendet und für `double`-Typen das Formatelement `%lf`. Bei `printf()` wird jeder `float`-Wert implizit in einen `double`-Wert konvertiert, somit spielt diese Unterscheidung bei `printf()` keine Rolle.



Die Funktion `scanf()` gilt heutzutage als unsicher. Aufgrund des Formatstrings und der Übergabe von Speicheradressen kann es zu unerlaubten Speicherzugriffen kommen, wenn falsche Eingaben geliefert werden. Mehr Informationen zu sicheren Ein- und Ausgabefunktionen können in Kapitel [➔ 16.10](#) nachgelesen werden.

### 16.9.1 Varianten von `scanf()`

Mit `scanf()` werden Werte formatiert eingelesen. Die Werte werden schlussendlich in Variablen gespeichert.

Bevor jedoch erklärt wird, wie Ausdrücke formatiert eingelesen werden können, werden die verschiedenen Varianten von `scanf()` vorgestellt:

Von dieser Funktion gibt es verschiedene Implementierungen, welche allesamt dieselben Formatierungen einlesen können, jedoch unterschiedliche Ein- und Ausgänge besitzen. Die entsprechenden Funktionen werden durch unterschiedliche Präfixe vor dem `scanf()` ausgedrückt:

|                      |                                                                                    |
|----------------------|------------------------------------------------------------------------------------|
| <code>scanf</code>   | Eingabe aus <code>stdin</code>                                                     |
| <code>fscanf</code>  | Eingabe aus Datei ( <code>f</code> = File)                                         |
| <code>sscanf</code>  | Eingabe aus String                                                                 |
| <code>vscanf</code>  | Eingabe aus <code>stdin</code> , Parameter aus variabler Parameterliste            |
| <code>vfscanf</code> | Eingabe aus Datei ( <code>f</code> = File), Parameter aus variabler Parameterliste |
| <code>vsscanf</code> | Eingabe aus String, Parameter aus variabler Parameterliste                         |

Die Funktion `scanf()` muss in der Lage sein, beliebig viele Werte einzulesen. Daher enthält der Prototyp von `scanf()` auch die Auslassung `(...)`. Genauso wie bei `printf()` können mit dem `v`-Präfix die aufzufüllenden Variablen anstelle der Auslassung mittels einer variablen Parameterliste mit dem Typ `va_list` angegeben werden. Hier werden diese Funktionen nicht weiter behandelt. Weitere Informationen zu variablen Parameterlisten und den Ellipsen können in Kapitel [→ 11.7](#) nachgelesen werden.

Das Präfix `f` sorgt dafür, dass die Eingabe nicht von der Standardeingabe `stdin` erfolgt, sondern von einem beliebigen File-Pointer. Folgendes ist der Prototyp der Funktion:

```
int fscanf(FILE* stream, const char* format, ...);
```

In C11 sind die beiden Parameter zusätzlich mit dem `restrict`-Schlüsselwort deklariert.

Das Präfix `s` sorgt dafür, dass die Eingabe von einem String erfolgt. Folgendes ist der Prototyp:

```
int sscanf(const char* s, const char* format, ...);
```



In C11 sind die beiden Pointer-Parameter wiederum zusätzlich mit dem `restrict`-Schlüsselwort deklariert.

Bei dieser Funktion werden die Werte für die aufzufüllenden Variablen aus dem in `s` gegebenen String extrahiert.

Es ist anzumerken, dass im Gegensatz zu den Schreib-Funktionen hier keine `sn`-Variante existiert. Während beim Schreiben in einen String der gegebene Puffer `s` ohne Angabe der Größe des Puffers überlaufen könnte, kann beim Lesen von einem `'\0'`-terminierten String ein ebensolcher Überlauf nicht stattfinden, da die Funktion nicht über das `'\0'`-Zeichen hinaus liest.

Die meisten Lesefunktionen erwarten explizit einen `'\0'`-terminierten String als Quelle. Handelt es sich bei der Quelle nicht um einen `'\0'`-terminierten String, so ist das Verhalten unbestimmt.



### 16.9.2 Formatiertes Einlesen von Werten

Genauso wie bei den Schreib-Funktionen hat die Funktion `scanf()` den Parameter `format`. Dieser Format-String enthält grundsätzlich genau dieselben Formatelemente mit dem Prozent-Zeichen `%` wie die Funktion `printf()`. Für genaue Erklärungen der Formatelemente wird somit auf die Beschreibung der Schreib-Funktionen in Kapitel [16.7.2](#) verwiesen.

Der wichtigste Unterschied zwischen `printf()` und `scanf()` zeigt sich bei den Parametern nach dem Format-String:

Während die Funktion `printf()` als Argumente bis auf den Format-String Ausdrücke mit Werten erwartet, erwartet die Funktion `scanf()` Adressen von Variablen.



```
int a;  
scanf("%d", &a);
```

Dies deswegen, da die Funktion `scanf()` den gelesenen Wert irgendwo speichern, also einem L-Wert zuweisen muss. Dies kann nur mittels Angabe einer Speicheradresse erreicht werden.

Beim Lesen von formatierten Werten muss man normalerweise nur angeben, von welchem Typ die Variable ist, in welcher der Wert gespeichert werden soll.

Stimmt ein Formatelement nicht mit dem Pointer-Typ des entsprechenden Argumentes überein, so kann das Programm abstürzen.



Des Weiteren hat `scanf()` ein spezielles Verhalten bei fehlerhaftem Lesen. Wird von der Eingabe ein nicht kompatibler Wert gelesen (beispielsweise ein String anstatt eines Integer), so resultiert ein Fehler. Wird bei einem Formatelement ein solch inkompatibler Wert gelesen, so beendet `scanf()` bei genau diesem Formatelement seine Tätigkeit und stellt das unverdauliche Zeichen zurück in die Standardeingabe.

Sollen also beispielsweise ein Integer `i` und eine Gleitpunkt-Zahl `x` über die Tastatur eingelesen werden, so wird folgende Zeile programmiert:

```
scanf("%d %e", &i, &x);
```

Erfolgt nun fälschlicherweise die Eingabe `"13 abc"`, so entspricht `"abc"` nicht dem erlaubten Format einer `float`-Zahl. In diesem Fall wird zwar in der Variablen `i` der erwartete Wert 13 stehen, der Inhalt von `x` ist jedoch undefiniert. Um solche Fälle aufzudecken, gibt `scanf()` als Rückgabewert die Anzahl der erfolgreich gelesenen Werte zurück. Der Rückgabewert von `scanf()` muss daher immer überprüft werden, wenn nicht ausgeschlossen werden kann, dass unpassende Eingaben kommen. Bei einer fehlerhaften Eingabe ist allerdings auch noch darauf zu achten, dass der Eingabepuffer mit den fehlerhaften Zeichen gelöscht wird, da sie ja nicht gelesen werden konnten und somit noch nicht verarbeitet wurden.

Als Rückgabewert gibt die Funktion `scanf()` die Anzahl der erfolgreich eingelesenen Eingabefelder zurück. Geht das Einlesen beim ersten Formatelement schief, so ist 0 der Rückgabewert. Der Rückgabewert von `scanf()` muss stets überprüft werden, um nicht verträgliche Eingaben erkennen und behandeln zu können.



Steht im Format-String ein Whitespace-Zeichen wie beispielsweise ein Leerzeichen oder ein Newline-Zeichen '\n', so bedeutet es für `scanf()`, alle Whitespace-Zeichen, die von der Standardeingabe kommen, zu überlesen. `scanf()` liest diese Zeichen aus der Standardeingabe und wirft sie weg. Steht kein Leerzeichen da, so wird auch keines gelesen (oder überlesen).

Ein unvorsichtigerweise im Format-String an ungünstiger Stelle angegebenes Leerzeichen oder Newline-Zeichen kann zu unerwarteten Effekten führen.



Die Funktion `scanf()` ist nützlich für einfache Eingaben, kann jedoch sehr kompliziert werden, wenn komplexe Daten damit gelesen werden sollen. Die Vielzahl an Möglichkeiten der Funktion `scanf()` kann in diesem Buch nicht behandelt werden. Abschließend wird im Folgenden ein einfaches Programmbeispiel vorgestellt, welches einen einfachen Einstieg in das Einlesen von Parametern über die Standardeingabe ermöglicht:

scanf1.c

```
// MSVC meldet fuer veraltete, unsichere C-Funktionen wie
// scanf einen Fehler.
// Mittels folgender Zeile koennen diese Fehler ausgeschaltet
// werden:
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>

int main(void) {
    printf("Eingabe: ");

    int zahl;
    int anzahl = scanf("%d", &zahl);

    printf("%d Wert erfolgreich eingelesen.\n", anzahl);
    printf("Der Wert war %d.\n", zahl);
    return 0;
}
```

Der folgende Dialog wurde geführt:

```
Eingabe: 3
1 Wert erfolgreich eingelesen,
der Wert war 3.
```

## 16.10 Alternative Funktionen für mehr Sicherheit nach C11

„Der Programmierer weiß, was er tut“ – dieser Ansatz ist früher wie heute ein Kernpunkt der Philosophie der Programmiersprache C. Das Schreiben eines Programmes soll demnach durch die Sprache nicht eingeschränkt werden. Diese Tatsache eröffnet sehr viele Möglichkeiten, die in anderen Sprachen – wie beispielsweise in Java – nicht verfügbar sind. Allerdings stellt dieser Ansatz auch ein recht großes Sicherheitsproblem dar. Aus Unachtsamkeit können sehr leicht unbeabsichtigte Pufferüberläufe auftreten. Sie sind häufig das Ergebnis, wenn wichtige Abfragen vergessen gehen, etwas übersehen wird oder schlicht und einfach jemand eben nicht ganz so genau wusste, was zu tun war. Die Auswirkungen können ganz harmlos oder aber sehr schwerwiegend sein, wie beispielsweise der Absturz des Systems oder das Löschen wichtiger Daten. Sie lassen sich von vornherein nicht vorhersagen. Pufferüberläufe können außerdem von außen ausgenutzt werden, um Angriffe auf das System zu starten, was in Zeiten des Internets keine Seltenheit mehr ist.

Viele Funktionen zur Datei- und String-Manipulation der Standardbibliothek in C vertrauten früher darauf, dass man beim Einlesen Arrays bereitstellte, die groß genug für die Anzahl der einzulesenden Zeichen waren. Die Standardbibliothek stellte zudem wie beispielsweise bei der Funktion `gets()` keine Möglichkeiten bereit, diesen Umstand zu überprüfen. Die Überprüfung musste selbst vorgenommen werden.

Ein guter Programmierstil war es, wenn die Größe der char-Arrays so gewählt wurde, dass dieser Puffer für den normalen Gebrauch ausreichte. Waren diese Arrays allerdings doch nicht groß genug, dann wurden Daten über das Array hinaus in den Speicher geschrieben und damit eventuell andere Daten oder gar Programmanweisungen überschrieben. Das Programm erfuhr nie etwas davon, wenn es tatsächlich zum Pufferüberlauf kam und hatte daher keine Möglichkeit, den Anwender zu warnen oder diesen Fehler irgendwie zu behandeln. Ein solch nachlässiger Programmierstil wurde früher von der Standardbibliothek indirekt gefördert, da viele Funktionen keine Überprüfungsmöglichkeiten bereitstellten, um solche Pufferüberläufe zu verhindern.

Viele alte Funktionen der C-Standardbibliothek konnten zu Sicherheitsproblemen führen. Beispielsweise verhinderte es die Funktion `gets()` in keinsten Weise, dass es zu Pufferüberläufen kommen konnte.



Der Microsoft Visual C Compiler meldet deswegen für unsichere Funktionen mittlerweile einen Fehler. Um diese Fehler zu umgehen und die ursprünglich für C standardisierten Funktionen nutzen zu können, kann in der ersten Zeile einer Implementationsdatei das Makro `_CRT_SECURE_NO_WARNINGS` definiert werden. Dann wird der Compiler keine Fehler generieren.



Die ursprünglich definierten Funktionen sind grundsätzlich immer noch verfügbar, existieren aber nur noch zur Kompatibilität mit früherem Code. Heutige C-Standards definieren neue, sichere Funktionen.

Um beispielsweise Pufferüberläufe zu verhindern, führt C11 die sogenannten „**bounds checking interfaces**“ ein. Es handelt sich hierbei um eine optionale Erweiterung, die eine Sammlung von alternativen **sicheren Funktionen** zu bereits existierenden unsicheren Funktionen der Standardbibliothek bereitstellt. Diese neuen Funktionen haben alle ein abschließendes `_s` am Ende ihres Namens, um sie von den normalen Funktionen unterscheiden zu können.



Diese neuen Funktionen sollen ein sicheres Programmieren fördern, indem sie beispielsweise die Puffergröße prüfen und Fehlermeldungen ausgeben, wenn der Puffer nicht groß genug ist. Damit können Pufferüberläufe einfacher vermieden werden. Diese und weitere Vorteile bieten die rund 60 neuen Funktionen beispielsweise für die Bereiche Ein- und Ausgabe, das Arbeiten mit Streams oder Zeitfunktionen. Hier in diesem Buch werden sie nur konzeptionell angesprochen.

Da die bounds checking interfaces eine optionale Erweiterung in C11 sind, existiert ein Makro, über das sich prüfen lässt, ob die neuen sicheren Funktionen überhaupt verfügbar sind. Ist das Makro `__STDC_WANT_LIB_EXT1__` als 1 definiert, dann sind diese Funktionen enthalten, ist es aber als 0 definiert, dann sind sie nicht in der Bibliothek enthalten. Ist dieses Makro dagegen gar nicht definiert, dann ist es implementierungsabhängig, ob diese neuen Sicherheitsfunktionen in der Standardbibliothek enthalten sind oder nicht.

Das Benutzen einer alternativen sicheren `_s`-Funktion anstatt ihres unsicheren Gegenparts aus der Standardbibliothek bringt verschiedene Vorteile mit sich. Die wichtigsten werden hier aufgeführt:

- Pufferüberläufe werden verhindert.
- Ergebnisstrings erhalten immer ein abschließendes `'\0'`-Zeichen.
- Zugriffsrechte können eingeschränkt werden.
- Rückgabewerte sind einheitlich vom Typ `errno_t` und geben Aufschluss über den Erfolg der durchgeführten Operation.

Jede Funktion mit `_s` am Ende kann ihr jeweiliges Gegenstück (ohne `_s`) ersetzen.



Hierbei müssen nur ein bis zwei Zeilen Code geändert werden.

### 16.10.1 Die Funktionen `gets()` und `gets_s()`

Die Standardbibliotheksfunktion `gets()` ist nach C90 folgendermaßen deklariert:

```
char* gets(char* s);
```

Sie liest eine Zeile aus der Standard-Eingabe (`stdin`) ein, bis sie auf ein Newline-Zeichen (`'\n'`) oder ein End of File stößt (EOF), und schreibt sie in einen Puffer, der vom Aufrufer zur Verfügung gestellt wird. Diese Funktion kennt allerdings ihre maximal verfügbare Puffergröße nicht. Dies stellt ein Sicherheitsproblem dar, da es sehr leicht zu Pufferüberläufen kommen kann. Aus diesem Grund gilt `gets()` als unsichere Funktion, wird seit C99 als veraltet eingestuft und ist mit C11 schließlich aus dem Standard entfernt worden. Ersetzt wird sie durch die Funktion `gets_s()`:

```
char* gets_s(char* s, rsize_t n);
```

Die `gets_s()`-Funktion arbeitet im Prinzip fast genauso wie ihr Vorgänger `gets()`. Sie liest allerdings nur maximal  $n - 1$  Zeichen ein, da ihr Puffer  $n - 1$  Zeichen plus das anzuhängende Zeichen `'\0'` aufnehmen kann. Ist die maximale Anzahl einzulesender Zeichen erreicht, dann wird der Lesevorgang beendet, auch wenn das Zeilenende noch nicht erreicht ist.



Wird das Zeilenende erreicht, dann wird das Newline-Zeichen nicht mit in den Puffer geschrieben. Nachdem das letzte Zeichen eingelesen wurde, wird ein abschließendes `'\0'`-Zeichen ans Ende des Puffers eingefügt.

Die Funktion `gets_s()` ist allerdings Teil der optionalen Erweiterung von C11. Als Alternative wird im Standard auf die `fgets()`-Funktion verwiesen:

```
char* fgets(char* s, int n, FILE* stream);
```

Diese Funktion verhält sich ganz ähnlich wie `gets_s()`. Sie liest statt von der Standard-Eingabe von einer Datei ein und schreibt – im Gegensatz zu `gets_s()` – dabei auch das Newline-Zeichen in den Puffer, falls es vorkommt. Will man von der Tastatur einlesen, so muss man einfach `stdin` als File-Pointer angeben.

### 16.10.2 Die Funktion `fopen_s()`

Eine weitere Funktion aus den bounds checking interfaces ist die Funktion `fopen_s()`:

```
errno_t fopen_s(  
    FILE* restrict * restrict streamptr,  
    const char* restrict filename,  
    const char* restrict mode);
```

Im Prinzip macht die Funktion `fopen_s()` genau dasselbe wie die Funktion `fopen()`, sie öffnet eine Datei. Der Pointer `streamptr` auf den Stream wird der Funktion als Parameter übergeben. Konnte die Datei erfolgreich geöffnet werden, dann zeigt `streamptr` auf die neu geöffnete Datei, ansonsten ist dieser Pointer NULL. Gibt die Funktion eine 0 zurück, dann war das Öffnen der Funktion erfolgreich. Für den Parameter `mode` sind alle Modi zulässig, die auch für die `fopen()`-Funktion gelten.

## 16.11 Übungsaufgaben

Hinweis: Wird bei der Funktion `fopen()` als Dateiname ein String wie `"test.dat"` angegeben, so wird die Datei im aktuellen Arbeitsverzeichnis geöffnet. Das aktuelle Arbeitsverzeichnis wird normalerweise von der Entwicklungsumgebung festgelegt oder durch die Programmierumgebung. Da diese je nach Situation unterschiedlich sein kann, empfiehlt sich daher, für die Übungen absolute Pfade anzugeben, beispielsweise `"C:/cbuch/test.dat"`.

### Aufgabe 1: Formatiert auf Festplatte schreiben und von der Festplatte lesen

- a) Schreiben Sie ein Programm, das `n` `int`-Zahlen als String in eine Datei schreibt. Schreiben Sie jeweils eine Zahl pro Zeile.
- b) Schreiben Sie ein zweites Programm, das diese Datei wieder einliest und den Durchschnitt der Zahlen berechnet.

### Aufgabe 2: Zeilenweise lesen.

Schreiben Sie ein Programm, welches eine beliebige Textdatei öffnet und die Anzahl darin enthaltener Zeilen zählt. Nutzen Sie hierfür die Funktionen `fgets()` und `feof()`.

Beachten Sie beim Testen, dass `fgets()` auch leere Zeilen (beispielsweise am Ende der Datei) erfasst.



**Aufgabe 3: Blockweise Ein- und Ausgabe.**

Lassen Sie folgendes Programm laufen. Es erzeugt eine Binärdatei, in welche 7 int-Zahlen und 3 float-Zahlen geschrieben werden.

binaerDatei.c

```
// MSVC meldet fuer veraltete, unsichere C-Funktionen wie
// fopen einen Fehler.
// Mittels folgender Zeile koennen diese Fehler ausgeschaltet
// werden:
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>
#define iN 7
#define fN 3

int iArr[iN] = { 1, 2, 3, 4, 5, 6, 7 };
float fArr[fN] = { 1.1f, 2.2f, 3.3f };

int main(void) {
    FILE* fp = fopen("test.dat", "wb");
    if (fp == NULL) {
        printf("Kann Datei nicht oeffnen.\n");
    } else {
        fwrite(iArr, sizeof(int), iN, fp);
        fwrite(fArr, sizeof(float), fN, fp);
        fclose(fp);
    }
    return 0;
}
```

Schreiben Sie ein Programm, welches diese Datei öffnet, alle Zahlen liest und sie auf dem Bildschirm ausgibt.

**Aufgabe 4: Dateien, Strukturen und formatierte Ausgabe**

Gegeben seien Daten, die der Verwaltung von Lagerbeständen dienen. Eine Liste solcher Daten sieht typischerweise folgendermaßen aus:

| Artikelnummer | Lagerbestand | Gewicht je Stück | Preis |
|---------------|--------------|------------------|-------|
| 9169          | 500          | 6.334            | 19.47 |
| 4464          | 962          | 29.358           | 15.48 |
| 9961          | 827          | 23.281           | 28.15 |
| 5436          | 827          | 11.942           | 4.00  |

- Schreiben Sie eine Struktur, welche diese Datenstruktur aufnimmt und füllen Sie die oben angegebenen Daten (die 4 Datenreihen) in ein Array ein.
- Geben Sie den Inhalt dieses Arrays auf dem Bildschirm aus mit folgendem Format: Artikelnummer linksbündig, maximal 8 Zeichen, Lagerbestand rechtsbündig, Gewicht rechtsbündig mit stets drei Nachkommastellen, Preis rechtsbündig mit stets zwei Nachkommastellen und dem Zusatz „EUR“.
- Schreiben Sie den Inhalt dieses Arrays in eine Datei. Nutzen Sie hierfür keine String-Funktionen!
- Schreiben Sie dann eine Funktion zu folgendem Prototyp:

```
void transferInDollar(  
    const char* dateiNameIn,  
    const char* dateiNameOut);
```

Diese Funktion öffnet die gegebene Eingabedatei und konvertiert alle Preise von Euro in Dollar und schreibt die Daten in die gegebene Ausgabedatei.

Gehen Sie davon aus, dass es stets 4 Datenreihen sind.

- Überlegen Sie sich, wie man Daten lesen könnte, wenn nicht von vornherein klar ist, wieviele Daten sich in der Datei befinden.
- Könnte man diese Daten auch mittels Stringfunktionen speichern? Was für Probleme treten möglicherweise auf?